Jupyter Notebook Execution Report

Name: Your Name

Date: February 03, 2026

Cell 1: ■ Markdown

Department of Computer Science and Engineering (Data Science)

S.Y. B.Tech. Sem: IV Subject: Statistics for Data Science (DJS23DPC253L)

Experiment 1A: Data Structures in Python

Date: February 3, 2026

Experiment Title: Data Structures in Python

Aim: To study data types in Python and its functions.

Software: Google Colab / Jupyter Notebook

Cell 2: ■ Markdown

Theory

Data structures in Python are specialized ways of organizing and storing data so that it can be accessed, modified, and managed efficiently. Python provides built-in data structures such as:

1. **Lists**: Ordered and mutable collections - allow duplicate elements

2. **Tuples**: Ordered but immutable data - cannot be changed after creation

3. **Dictionaries**: Fast storage and retrieval of data using key–value pairs

4. **Sets**: Unordered collection of unique elements

Choosing the appropriate data structure helps optimize performance, improves code readability, and enables effective problem-solving in programming.

Cell 3: ■ Markdown

Implementation Questions and Solutions

COMMENTS

Cell 4: ■ Markdown

Q1. Write a comment in Python.

Cell 5: ■ Code

```
# This is a single-line comment in Python

# Comments are used to explain code and make it more readable

print("Hello, World!") # This is an inline comment
```

Output:

```
Hello, World!
```

Cell 6: ■ Markdown

Q2. Write a multiline comment/paragraph in Python.

Cell 7: ■ Code

```
"""

This is a multiline comment in Python.

It can span across multiple lines.

Commonly used for docstrings to describe functions, classes, or modules.

Each line provides additional documentation.

"""

print("Multiline comments are very useful for documentation")
```

```
# Alternative method using triple single quotes

'''

This is another way to write multiline comments

using triple single quotes.

Both methods work similarly.

'''

print("Both methods work!")
```

Output:

```
Multiline comments are very useful for documentation
Both methods work!
```

Cell 8: ■ Markdown

PRIMITIVE DATA TYPES

Cell 9: ■ Markdown

Q3. Write a program to print an integer, float, string, complex number, Boolean, and bytes in Python and display their data type.

Cell 10: ■ Code

```
# Integer data type
integer_value = 42
print(f"Integer: {integer_value}")
print(f"Data type: {type(integer_value)}")
print()
```

```
# Float data type
float_value = 3.14159
print(f"Float: {float_value}")
print(f"Data type: {type(float_value)}")
print()
```

```
# String data type
string_value = "Hello, Python!"
print(f"String: {string_value}")
print(f"Data type: {type(string_value)}")
print()
```

```
# Complex number data type
complex_value = 3 + 4j
print(f"Complex Number: {complex_value}")
print(f"Data type: {type(complex_value)}")
```

```
print()
```

```
# Boolean data type
boolean_value = True
print(f"Boolean: {boolean_value}")
print(f"Data type: {type(boolean_value)}")
print()
```

```
# Bytes data type
bytes_value = b"Hello"
print(f"Bytes: {bytes_value}")
print(f"Data type: {type(bytes_value)}")
```

Output:

```
Integer: 42
Data type: &lt;class 'int'&gt;

Float: 3.14159
Data type: &lt;class 'float'&gt;

String: Hello, Python!
Data type: &lt;class 'str'&gt;

Complex Number: (3+4j)
Data type: &lt;class 'complex'&gt;

Boolean: True
Data type: &lt;class 'bool'&gt;

Bytes: b'Hello'
Data type: &lt;class 'bytes'&gt;
```

Cell 11: ■ Markdown

DATA STRUCTURES: LISTS

Cell 12: ■ Markdown

Q4. Write a program to create a list. Collect heterogenous data in it.

Cell 13: ■ Code

```
# Create a list with heterogeneous data types
heterogeneous_list = [42, 3.14, "Python", True, [1, 2, 3], {"name": "John"}]
print("Heterogeneous List:")
print(heterogeneous_list)
print(f"\nList contains: integers, floats, strings, booleans, nested lists, and dictionaries")
```

Output:

```
Heterogeneous List:
[42, 3.14, 'Python', True, [1, 2, 3], {'name': 'John'}]

List contains: integers, floats, strings, booleans, nested lists, and dictionaries
```

Cell 14: ■ Markdown

Q5. Write a program to print a list.

Cell 15: ■ Code

```
# Create and print a list
fruits = ["apple", "banana", "cherry", "date", "elderberry"]
print("Original List:")
print(fruits)
```

```
# Print each element on a new line
print("\nElements one by one:")
for fruit in fruits:
print(fruit)
```

Output:

```
Original List:
['apple', 'banana', 'cherry', 'date', 'elderberry']

Elements one by one:
apple
banana
cherry
date
```

```
elderberry
```

Cell 16: ■ Markdown

Q6. Write a program to print a new list. Append an item in this list.

Cell 17: ■ Code

```
# Create a new list
colors = ["red", "blue", "green"]
print("Original List:")
print(colors)
```

```
# Append a new item
colors.append("yellow")
print("\nList after appending 'yellow':")
print(colors)
```

Output:

```
Original List:
['red', 'blue', 'green']
List after appending 'yellow':
['red', 'blue', 'green', 'yellow']
```

Cell 18: ■ Markdown

Q7. Write a program to make a copy of the previous list.

Cell 19: ■ Code

```
# Make a copy of the list
original_list = ["red", "blue", "green", "yellow"]
print("Original List:")
print(original_list)
```

```
# Method 1: Using copy() method
copied_list1 = original_list.copy()
print("\nCopied List (using copy() method):")
```

```
print(copied_list1)
```

```
# Method 2: Using list() constructor
copied_list2 = list(original_list)
print("\nCopied List (using list() constructor):")
print(copied_list2)
```

```
# Method 3: Using slicing
copied_list3 = original_list[:]
print("\nCopied List (using slicing):")
print(copied_list3)
```

Output:

```
Original List:
['red', 'blue', 'green', 'yellow']

Copied List (using copy() method):
['red', 'blue', 'green', 'yellow']

Copied List (using list() constructor):
['red', 'blue', 'green', 'yellow']

Copied List (using slicing):
['red', 'blue', 'green', 'yellow']
```

Cell 20: ■ Markdown

Q8. Write a program to concatenate 2 lists and print the output.

Cell 21: ■ Code

```
# Create two lists
list1 = [1, 2, 3, 4]
list2 = [5, 6, 7, 8]
print("List 1:", list1)
print("List 2:", list2)
```

```
# Method 1: Using + operator
concatenated_list1 = list1 + list2
print("\nConcatenated List (using + operator):")
```

```
print(concatenated_list1)
```

```
# Method 2: Using extend() method
concatenated_list2 = list1.copy()
concatenated_list2.extend(list2)
print("\nConcatenated List (using extend()):")
print(concatenated_list2)
```

Output:

```
List 1: [1, 2, 3, 4]
List 2: [5, 6, 7, 8]

Concatenated List (using + operator):
[1, 2, 3, 4, 5, 6, 7, 8]

Concatenated List (using extend()):
[1, 2, 3, 4, 5, 6, 7, 8]
```

Cell 22: ■ Markdown

Q9. Write a program to count the number of elements present in a list.

Cell 23: ■ Code

```
# Create a list and count elements
numbers = [10, 20, 30, 40, 50, 60, 70]
print("List:", numbers)
```

```
# Count specific element
count_30 = numbers.count(30)
print(f"Count of 30: {count_30}")
```

```
# Count with duplicate values
list_with_duplicates = [5, 5, 5, 10, 15, 5, 20]
print(f"\nList with duplicates: {list_with_duplicates}")
count_5 = list_with_duplicates.count(5)
print(f"Count of 5: {count_5}")
```

Output:

```
List: [10, 20, 30, 40, 50, 60, 70]
```

```
Count of 30: 1

List with duplicates: [5, 5, 5, 10, 15, 5, 20]
Count of 5: 4
```

Cell 24: ■ Markdown

Q10. Write a program to print the length of a list.

Cell 25: ■ Code

```
# Print length of different lists
fruits = ["apple", "banana", "cherry", "date"]
print(f"Fruits list: {fruits}")
print(f"Length: {len(fruits)}")
```

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(f"\nNumbers list: {numbers}")
print(f"Length: {len(numbers)}")
```

```
empty_list = []
print(f"\nEmpty list: {empty_list}")
print(f"Length: {len(empty_list)}")
```

Output:

```
Fruits list: ['apple', 'banana', 'cherry', 'date']
Length: 4

Numbers list: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Length: 10

Empty list: []
Length: 0
```

Cell 26: ■ Markdown

Q11. Write a program to append more than 1 item in a list.

Cell 27: ■ Code

```
# Create a list and append multiple items
my_list = [1, 2, 3]
print("Original list:", my_list)
```

```
# Append multiple items one by one
my_list.append(4)
my_list.append(5)
my_list.append(6)
print("\nAfter appending 4, 5, 6:")
print(my_list)
```

```
# Alternative: Using extend to add multiple items at once
my_list.extend([7, 8, 9, 10])
print("\nAfter extending with [7, 8, 9, 10]:")
print(my_list)
```

Output:

```
Original list: [1, 2, 3]

After appending 4, 5, 6:
[1, 2, 3, 4, 5, 6]

After extending with [7, 8, 9, 10]:
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Cell 28: ■ Markdown

Q12. Write a program to extend a list.

Cell 29: ■ Code

```
# Create a list and extend it
list1 = ["a", "b", "c"]
list2 = ["d", "e", "f"]
list3 = ["g", "h", "i"]
```

```
print("List 1:", list1)
print("List 2:", list2)
print("List 3:", list3)
```

```
# Extend list1 with list2
list1.extend(list2)
print("\nAfter extending list1 with list2:")
print("List 1:", list1)
```

```
# Extend list1 with list3
list1.extend(list3)
print("\nAfter extending list1 with list3:")
print("List 1:", list1)
```

Output:

```
List 1: ['a', 'b', 'c']
List 2: ['d', 'e', 'f']
List 3: ['g', 'h', 'i']

After extending list1 with list2:
List 1: ['a', 'b', 'c', 'd', 'e', 'f']

After extending list1 with list3:
List 1: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']
```

Cell 30: ■ Markdown

Q13. Write a program to insert a value at a position in a list.

Cell 31: ■ Code

```
# Create a list and insert values at specific positions
numbers = [10, 20, 30, 40, 50]
print("Original list:", numbers)
```

```
# Insert 15 at index 1
numbers.insert(1, 15)
print("\nAfter inserting 15 at index 1:")
print(numbers)
```

```
# Insert 35 at index 4
numbers.insert(4, 35)
print("\nAfter inserting 35 at index 4:")
```

```
print(numbers)
```

```
# Insert at beginning (index 0)
numbers.insert(0, 5)
print("\nAfter inserting 5 at index 0:")
print(numbers)
```

Output:

```
Original list: [10, 20, 30, 40, 50]

After inserting 15 at index 1:
[10, 15, 20, 30, 40, 50]

After inserting 35 at index 4:
[10, 15, 20, 30, 35, 40, 50]

After inserting 5 at index 0:
[5, 10, 15, 20, 30, 35, 40, 50]
```

Cell 32: ■ Markdown

Q14. Write a program to delete a value at a given position in a list.

Cell 33: ■ Code

```
# Create a list and delete values at specific positions
colors = ["red", "blue", "green", "yellow", "purple"]
print("Original list:", colors)
```

```
# Delete element at index 2
del colors[2]
print("\nAfter deleting element at index 2:")
print(colors)
```

```
# Delete element at index 0
del colors[0]
print("\nAfter deleting element at index 0:")
print(colors)
```

```
# Delete last element
del colors[-1]
```

```
print("\nAfter deleting last element:")
print(colors)
```

Output:

```
Original list: ['red', 'blue', 'green', 'yellow', 'purple']

After deleting element at index 2:
['red', 'blue', 'yellow', 'purple']

After deleting element at index 0:
['blue', 'yellow', 'purple']

After deleting last element:
['blue', 'yellow']
```

Cell 34: ■ Markdown

Q15. Write a program to remove a value from the list.

Cell 35: ■ Code

```
# Create a list and remove specific values
fruits = ["apple", "banana", "cherry", "banana", "date"]
print("Original list:", fruits)
```

```
# Remove first occurrence of 'banana'
fruits.remove("banana")
print("\nAfter removing 'banana':")
print(fruits)
```

```
# Remove another value
fruits.remove("cherry")
print("\nAfter removing 'cherry':")
print(fruits)
```

```
# Try removing with pop() method
numbers = [10, 20, 30, 40, 50]
print("\nNumbers list:", numbers)
removed = numbers.pop(2) # Remove and return element at index 2
print(f"Removed element: {removed}")
```

```
print("Remaining list:", numbers)
```

Output:

```
Original list: ['apple', 'banana', 'cherry', 'banana', 'date']

After removing 'banana':
['apple', 'cherry', 'banana', 'date']

After removing 'cherry':
['apple', 'banana', 'date']

Numbers list: [10, 20, 30, 40, 50]
Removed element: 30
Remaining list: [10, 20, 40, 50]
```

Cell 36: ■ Markdown

Q16. Write a program to slice the data in a list.

Cell 37: ■ Code

```
# Create a list and slice it
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print("Original list:", numbers)
```

```
# Get first 5 elements
print("\nFirst 5 elements (numbers[:5]):", numbers[:5])
```

```
# Get elements from index 2 to 7 (exclusive of 7)
print("Elements from index 2 to 7 (numbers[2:7]):", numbers[2:7])
```

```
# Get elements with step
print("Every 2nd element (numbers[::2]):", numbers[::2])
```

```
# Get elements in reverse
print("Reversed list (numbers[::-1]):", numbers[::-1])
```

Output:

```
Original list: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

First 5 elements (numbers[:5]): [0, 1, 2, 3, 4]
Elements from index 2 to 7 (numbers[2:7]): [2, 3, 4, 5, 6]
```

```
Every 2nd element (numbers[::2]): [0, 2, 4, 6, 8]
Reversed list (numbers[::-1]): [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Cell 38: ■ Markdown

Q17. Write a program to slice data in a list using positions.

Cell 39: ■ Code

```python
# Create a list and demonstrate different slicing operations
items = ["item0", "item1", "item2", "item3", "item4", "item5", "item6", "item7"]
print("Original list:", items)
print("\nSlicing operations:")
```

```python
# From index 1 to 4
print(f"items[1:4]: {items[1:4]}")
```

```python
# From index 0 to 3
print(f"items[0:3]: {items[0:3]}")
```

```python
# From index 4 onwards
print(f"items[4:]: {items[4:]}")
```

```python
# From beginning to index 5
print(f"items[:5]: {items[:5]}")
```

```python
# With step
print(f"items[1:7:2]: {items[1:7:2]}")
```

Output:

```
Original list: ['item0', 'item1', 'item2', 'item3', 'item4', 'item5', 'item6', 'item7']
Slicing operations:
items[1:4]: ['item1', 'item2', 'item3']
items[0:3]: ['item0', 'item1', 'item2']
items[4:]: ['item4', 'item5', 'item6', 'item7']
items[:5]: ['item0', 'item1', 'item2', 'item3', 'item4']
items[1:7:2]: ['item1', 'item3', 'item5']
```

Cell 40: ■ Markdown

Q18. Write a program to print the last 8 elements.

Cell 41: ■ Code

```
# Create a list with at least 8 elements
numbers = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110]
print("Original list:", numbers)
print(f"Length: {len(numbers)}")
```

```
# Print last 8 elements using negative indexing
print("\nLast 8 elements (numbers[-8:]):")
print(numbers[-8:])
```

```
# Alternative method
print("\nUsing len():")
print(numbers[len(numbers)-8:])
```

Output:

```
Original list: [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110]
Length: 11

Last 8 elements (numbers[-8:]):
[40, 50, 60, 70, 80, 90, 100, 110]

Using len():
[40, 50, 60, 70, 80, 90, 100, 110]
```

Cell 42: ■ Markdown

Q19. Write a program to print the last value of a list.

Cell 43: ■ Code

```
# Create a list and print the last value
numbers = [10, 20, 30, 40, 50]
print("List:", numbers)
```

```
# Method 1: Using index -1
print(f"\nLast value (numbers[-1]): {numbers[-1]}")
```

```
# Method 2: Using len()
print(f"Last value (numbers[len(numbers)-1]): {numbers[len(numbers)-1]}")
```

```
# Method 3: Using pop() - removes the element
numbers_copy = numbers.copy()
last = numbers_copy.pop()
print(f"Last value (using pop()): {last}")
print(f"List after pop(): {numbers_copy}")
```

Output:

```
List: [10, 20, 30, 40, 50]

Last value (numbers[-1]): 50
Last value (numbers[len(numbers)-1]): 50
Last value (using pop()): 50
List after pop(): [10, 20, 30, 40]
```

Cell 44: ■ Markdown

Q20. Write a program to print the central value of a list.

Cell 45: ■ Code

```
# Create a list and print the central/middle value
numbers = [10, 20, 30, 40, 50]
print("List with odd length:", numbers)
middle_index = len(numbers) // 2
print(f"Central value: {numbers[middle_index]}")
```

```
# For list with even length
numbers_even = [10, 20, 30, 40, 50, 60]
print(f"\nList with even length: {numbers_even}")
middle_index_even = len(numbers_even) // 2
print(f"Middle indices: {middle_index_even-1} and {middle_index_even}")
print(f"Values at middle: {numbers_even[middle_index_even-1]} and
{numbers_even[middle_index_even]}")
print(f"Average of middle values: {(numbers_even[middle_index_even-1] +
numbers_even[middle_index_even]) / 2}")
```

Output:

```
List with odd length: [10, 20, 30, 40, 50]
Central value: 30

List with even length: [10, 20, 30, 40, 50, 60]
Middle indices: 2 and 3
Values at middle: 30 and 40
Average of middle values: 35.0
```

Cell 46: ■ Markdown

DATA STRUCTURES: TUPLES

Cell 47: ■ Markdown

Q21. Write a program to create a tuple. Collect heterogenous data in it.

Cell 48: ■ Code

```
# Create a tuple with heterogeneous data types
heterogeneous_tuple = (42, 3.14, "Python", True, [1, 2, 3], {"name": "John"})
print("Heterogeneous Tuple:")
print(heterogeneous_tuple)
print(f"\nTuple length: {len(heterogeneous_tuple)}")
print(f"Tuple is immutable - once created, it cannot be modified")
```

Output:

```
Heterogeneous Tuple:
(42, 3.14, 'Python', True, [1, 2, 3], {'name': 'John'})

Tuple length: 6
Tuple is immutable - once created, it cannot be modified
```

Cell 49: ■ Markdown

Q22. Write a program to print the position of an item in the tuple.

Cell 50: ■ Code

```
# Create a tuple and find position of items
fruits = ("apple", "banana", "cherry", "date", "elderberry")
print("Tuple:", fruits)
```

```
# Find position using index()
position_banana = fruits.index("banana")
print(f"\nPosition of 'banana': {position_banana}")
```

```
position_cherry = fruits.index("cherry")
print(f"Position of 'cherry': {position_cherry}")
```

```
position_apple = fruits.index("apple")
print(f"Position of 'apple': {position_apple}")
```

Output:

```
Tuple: ('apple', 'banana', 'cherry', 'date', 'elderberry')

Position of 'banana': 1
Position of 'cherry': 2
Position of 'apple': 0
```

Cell 51: ■ Markdown

Q23. Print a new tuple. Write a program to concatenate two tuples.

Cell 52: ■ Code

```
# Create two tuples
tuple1 = (1, 2, 3, 4)
tuple2 = (5, 6, 7, 8)
print("Tuple 1:", tuple1)
print("Tuple 2:", tuple2)
```

```
# Concatenate tuples using + operator
concatenated_tuple = tuple1 + tuple2
print("\nConcatenated Tuple:")
print(concatenated_tuple)
```

Output:

```
Tuple 1: (1, 2, 3, 4)
Tuple 2: (5, 6, 7, 8)

Concatenated Tuple:
(1, 2, 3, 4, 5, 6, 7, 8)
```

Cell 53: ■ Markdown

Q24. Write a program to print the value at position 2 in the concatenated tuple.

Cell 54: ■ Code

```python
# Create and concatenate tuples
tuple1 = (10, 20, 30, 40)
tuple2 = (50, 60, 70, 80)
concatenated_tuple = tuple1 + tuple2
print("Tuple 1:", tuple1)
print("Tuple 2:", tuple2)
print("\nConcatenated Tuple:", concatenated_tuple)
```

```python
# Print value at position/index 2
print(f"\nValue at index 2: {concatenated_tuple[2]}")
print(f"Value at index 0: {concatenated_tuple[0]}")
print(f"Value at index 5: {concatenated_tuple[5]}")
```

Output:

```
Tuple 1: (10, 20, 30, 40)
Tuple 2: (50, 60, 70, 80)

Concatenated Tuple: (10, 20, 30, 40, 50, 60, 70, 80)

Value at index 2: 30
Value at index 0: 10
Value at index 5: 60
```

Cell 55: ■ Markdown

Q25. Write a program to change the element of a tuple.

Cell 56: ■ Code

```
# Note: Tuples are immutable, so we cannot change elements directly
# But we can convert to list, modify, and convert back
```

```
original_tuple = (10, 20, 30, 40, 50)
print("Original Tuple:", original_tuple)
```

```
# Method 1: Convert to list, modify, convert back to tuple
temp_list = list(original_tuple)
temp_list[2] = 99 # Change element at index 2
modified_tuple = tuple(temp_list)
print("\nModified Tuple (changed index 2 to 99):", modified_tuple)
```

```
# Method 2: Create a new tuple with modified element
new_tuple = original_tuple[:2] + (100,) + original_tuple[3:]
print("\nNew Tuple (changed index 2 to 100):", new_tuple)
```

Output:

```
Original Tuple: (10, 20, 30, 40, 50)
Modified Tuple (changed index 2 to 99): (10, 20, 99, 40, 50)
New Tuple (changed index 2 to 100): (10, 20, 100, 40, 50)
```

Cell 57: ■ Markdown

DATA STRUCTURES: DICTIONARY

Cell 58: ■ Markdown

Q26. Write a program to create and print a dictionary.

Cell 59: ■ Code

```
# Create and print a dictionary
student = {
"name": "John Doe",
"roll_number": 101,
"email": "john@example.com",
```

```
"age": 20,
"gpa": 3.8
}
print("Student Dictionary:")
print(student)
```

```
# Print formatted
print("\nFormatted Output:")
for key, value in student.items():
print(f"{key}: {value}")
```

Output:

```
Student Dictionary:
{'name': 'John Doe', 'roll_number': 101, 'email': 'john@example.com', 'age': 20, 'gpa': 3.8}

Formatted Output:
name: John Doe
roll_number: 101
email: john@example.com
age: 20
gpa: 3.8
```

Cell 60: ■ Markdown

Q27. Write a program to print values of a dictionary using keys.

Cell 61: ■ Code

```
# Create a dictionary
person = {
"name": "Alice",
"age": 25,
"city": "New York",
"profession": "Engineer"
}
print("Dictionary:", person)
```

```
# Access values using keys
```

```
print(f"\nName: {person['name']}")
print(f"Age: {person['age']}")
print(f"City: {person['city']}")
print(f"Profession: {person['profession']}")
```

```
# Using get() method
print(f"\nUsing get() method:")
print(f"Age: {person.get('age')}")
print(f"Salary (not in dict): {person.get('salary', 'Not available')}")
```

Output:

```
Dictionary: {'name': 'Alice', 'age': 25, 'city': 'New York', 'profession': 'Engineer'}

Name: Alice
Age: 25
City: New York
Profession: Engineer

Using get() method:
Age: 25
Salary (not in dict): Not available
```

Cell 62: ■ Markdown

Q28. Write a program to create a multidimensional dictionary.

Cell 63: ■ Code

```
# Create a multidimensional (nested) dictionary
company = {
"employee1": {
"name": "John",
"department": "IT",
"salary": 50000
},
"employee2": {
"name": "Sarah",
"department": "HR",
```

```
"salary": 45000
},
"employee3": {
"name": "Mike",
"department": "Finance",
"salary": 55000
}
}
```

```
print("Multidimensional Dictionary:")
print(company)
```

```
# Print in formatted way
print("\nFormatted Output:")
for emp_id, emp_details in company.items():
print(f"\n{emp_id}:")
for key, value in emp_details.items():
print(f" {key}: {value}")
```

Output:

```
Multidimensional Dictionary:
{'employee1': {'name': 'John', 'department': 'IT', 'salary': 50000}, 'employee2': {'name': 'Sarah

Formatted Output:

employee1:
  name: John
  department: IT
  salary: 50000

employee2:
  name: Sarah
  department: HR
  salary: 45000

employee3:
  name: Mike
  department: Finance
  salary: 55000
```

Cell 64: ■ Markdown

Q29. Write a program to print values from the multidimensional dictionary using keys.

Cell 65: ■ Code

```
# Use the multidimensional dictionary from Q28
company = {
"employee1": {
"name": "John",
"department": "IT",
"salary": 50000
},
"employee2": {
"name": "Sarah",
"department": "HR",
"salary": 45000
}
}
```

```
# Access values using nested keys
print("Accessing nested dictionary values:")
print(f"Employee 1 name: {company['employee1']['name']}")
print(f"Employee 1 department: {company['employee1']['department']}")
print(f"Employee 1 salary: {company['employee1']['salary']}")
```

```
print(f"\nEmployee 2 name: {company['employee2']['name']}")
print(f"Employee 2 department: {company['employee2']['department']}")
print(f"Employee 2 salary: {company['employee2']['salary']}")
```

Output:

```
Accessing nested dictionary values:
Employee 1 name: John
Employee 1 department: IT
Employee 1 salary: 50000

Employee 2 name: Sarah
```

```
Employee 2 department: HR
Employee 2 salary: 45000
```

Cell 66: ■ Markdown

CONTROL FLOW: IF-ELSE CONDITION

Cell 67: ■ Markdown

Q30. Brother is 12 years old. Sister is 15 years old. Write a program that prints who is older using if-else statement.

Cell 68: ■ Code

```
# Fixed ages
brother_age = 12
sister_age = 15
```

```
print(f"Brother's age: {brother_age}")
print(f"Sister's age: {sister_age}")
print()
```

```
# Using if-else statement
if brother_age &gt; sister_age:
print("Brother is older")
elif sister_age &gt; brother_age:
print("Sister is older")
else:
print("Both are of the same age")
```

Output:

```
Brother's age: 12
Sister's age: 15

Sister is older
```

Cell 69: ■ Markdown

Q31. Take the input of ages from the user. Write a program that prints who is older using if-else statement.

Cell 70: ■ Code

```
# Take input from user

brother_age = int(input("Enter brother's age: "))

sister_age = int(input("Enter sister's age: "))
```

```
print(f"\nBrother's age: {brother_age}")

print(f"Sister's age: {sister_age}")

print()
```

```
# Using if-else statement

if brother_age &gt; sister_age:

print(f"Brother is older by {brother_age - sister_age} year(s)")

elif sister_age &gt; brother_age:

print(f"Sister is older by {sister_age - brother_age} year(s)")

else:

print("Both are of the same age")
```

Output:

```
Enter brother's age:
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\admin\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "&lt;string&gt;", line 14
    else:
         ^
IndentationError: expected an indented block after 'else' statement on line 14
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "c:\Users\admin\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 408
    exec(source, glb)
    ~~~~^^^^^^^^^^^^^
  File "&lt;string&gt;", line 2, in &lt;module&gt;
EOFError: EOF when reading a line
```

Cell 71: ■ Markdown

LOOPS: FOR LOOP

Cell 72: ■ Markdown

Q32. Write a program that prints the elements of a list using for loop.

Cell 73: ■ Code

```
# Create a list
fruits = ["apple", "banana", "cherry", "date", "elderberry"]
print("List of fruits:")
print(fruits)
print("\nElements using for loop:")
```

```
# Print elements using for loop
for fruit in fruits:
print(fruit)
```

Output:

```
List of fruits:
['apple', 'banana', 'cherry', 'date', 'elderberry']

Elements using for loop:
apple
banana
cherry
date
elderberry
```

Cell 74: ■ Markdown

Q33. Write a program that enumerates and prints the elements of a list using for loop.

Cell 75: ■ Code

```
# Create a list
colors = ["red", "green", "blue", "yellow", "purple"]
print("List of colors:")
```

```
print(colors)
print("\nEnumerated elements:")
```

```
# Print with enumeration
for index, color in enumerate(colors):
print(f"Index {index}: {color}")
```

Output:

```
List of colors:
['red', 'green', 'blue', 'yellow', 'purple']

Enumerated elements:
Index 0: red
Index 1: green
Index 2: blue
Index 3: yellow
Index 4: purple
```

Cell 76: ■ Markdown

FUNCTIONS

Cell 77: ■ Markdown

Q34. Write a program to create a function.

Cell 78: ■ Code

```
# Create a simple function
def greet():
"""A simple function that greets the user"""
print("Hello! Welcome to Python Functions!")
print("This is a simple function without parameters.")
```

```
# Call the function
greet()
```

Output:

```
Hello! Welcome to Python Functions!
```

```
This is a simple function without parameters.
```

Cell 79: ■ Markdown

Q35. Create a function that adds two numbers.

Cell 80: ■ Code

```python
# Create a function that adds two numbers
def add(a, b):
"""Function to add two numbers and return the result"""
result = a + b
return result
```

```python
# Call the function with different numbers
print("Function to add two numbers:")
print(f"add(10, 20) = {add(10, 20)}")
print(f"add(5, 15) = {add(5, 15)}")
print(f"add(100, 200) = {add(100, 200)}")
```

Output:

```
Function to add two numbers:
add(10, 20) = 30
add(5, 15) = 20
add(100, 200) = 300
```

Cell 81: ■ Markdown

Q36. Create a function that adds two numbers. Take input from the user.

Cell 82: ■ Code

```python
# Create a function that adds two numbers
def add_numbers():
"""Function to take input from user and add two numbers"""
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
```

```
sum_result = num1 + num2
print(f"\nSum of {num1} and {num2} is: {sum_result}")
```

```
# Call the function
add_numbers()
```

Output:

```
Enter first number:
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\admin\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403
    result = eval(lines[-1], glb)
  File "&lt;string&gt;", line 1, in &lt;module&gt;
  File "&lt;string&gt;", line 4, in add_numbers
EOFError: EOF when reading a line
```

Cell 83: ■ Markdown

Q37. Create a function that adds two strings. Take input from the user.

Cell 84: ■ Code

```
# Create a function that concatenates two strings
def add_strings():
"""Function to concatenate two strings from user input"""
str1 = input("Enter first string: ")
str2 = input("Enter second string: ")
concatenated = str1 + str2
print(f"\nConcatenation of '{str1}' and '{str2}' is: '{concatenated}'")
```

```
# Call the function
add_strings()
```

Output:

```
Enter first string:
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\admin\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403
    result = eval(lines[-1], glb)
  File "&lt;string&gt;", line 1, in &lt;module&gt;
  File "&lt;string&gt;", line 4, in add_strings
```

```
EOFError: EOF when reading a line
```

Cell 85: ■ Markdown

DATA STRUCTURES: SETS

Cell 86: ■ Markdown

Q38. Write a program to create and print a set.

Cell 87: ■ Code

```
# Create and print a set
fruits = {"apple", "banana", "cherry", "date", "elderberry"}
print("Set of fruits:")
print(fruits)
print(f"\nSet data type: {type(fruits)}")
print(f"Set length: {len(fruits)}")
print("Sets are unordered collections of unique elements.")
```

Output:

```
Set of fruits:
{'date', 'cherry', 'apple', 'banana', 'elderberry'}

Set data type: &lt;class 'set'&gt;
Set length: 5
Sets are unordered collections of unique elements.
```

Cell 88: ■ Markdown

Q39. Write a program to print a set with duplicate values.

Cell 89: ■ Code

```
# Create a set with duplicate values
# Note: Sets automatically remove duplicates
numbers = {1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5}
print("Set created with duplicate values:")
```

```
print(f"Original input: {{1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5}}")
print(f"\nSet output (duplicates removed):")
print(numbers)
print(f"\nNote: Sets automatically eliminate duplicate elements")
```

Output:

```
Set created with duplicate values:
Original input: {1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5}

Set output (duplicates removed):
{1, 2, 3, 4, 5}

Note: Sets automatically eliminate duplicate elements
```

Cell 90: ■ Markdown

Q40. Write a program to print the length of a set.

Cell 91: ■ Code

```
# Create sets and print their lengths
set1 = {10, 20, 30, 40, 50}
print(f"Set 1: {set1}")
print(f"Length of Set 1: {len(set1)}")
```

```
set2 = {"a", "b", "c", "d", "e", "f", "g"}
print(f"\nSet 2: {set2}")
print(f"Length of Set 2: {len(set2)}")
```

```
empty_set = set() # Create an empty set
print(f"\nEmpty Set: {empty_set}")
print(f"Length of Empty Set: {len(empty_set)}")
```

Output:

```
Set 1: {50, 20, 40, 10, 30}
Length of Set 1: 5

Set 2: {'b', 'a', 'd', 'c', 'g', 'f', 'e'}
Length of Set 2: 7

Empty Set: set()
```

```
Length of Empty Set: 0
```

Cell 92: ■ Markdown

Q41. Write a program to create a set and print its data type.

Cell 93: ■ Code

```
# Create a set and print its data type
my_set = {42, 3.14, "hello", (1, 2)}
print(f"Set: {my_set}")
print(f"Data type: {type(my_set)}")
print(f"Type name: {type(my_set).__name__}")
```

```
# Check data type of different collections
print("\nComparison of data types:")
print(f"List [1, 2, 3] type: {type([1, 2, 3])}")
print(f"Tuple (1, 2, 3) type: {type((1, 2, 3))}")
print(f"Set {{1, 2, 3}} type: {type({1, 2, 3})}")
print(f"Dictionary {{'a': 1}} type: {type({'a': 1})}")
```

Output:

```
Set: {(1, 2), 42, 3.14, 'hello'}
Data type: &lt;class 'set'&gt;
Type name: set

Comparison of data types:
List [1, 2, 3] type: &lt;class 'list'&gt;
Tuple (1, 2, 3) type: &lt;class 'tuple'&gt;
Set {1, 2, 3} type: &lt;class 'set'&gt;
Dictionary {'a': 1} type: &lt;class 'dict'&gt;
```

Cell 94: ■ Markdown

Q42. Write a program to check if a set takes duplicate values with different capitalization/formatting.

Cell 95: ■ Code

```
# Check duplicate values with different capitalization
set_with_caps = {"apple", "Apple", "APPLE", "Banana", "banana", "BANANA"}
print("Set with different capitalizations:")
print(f"Input: {{'apple', 'Apple', 'APPLE', 'Banana', 'banana', 'BANANA'}}")
print(f"\nOutput: {set_with_caps}")
print(f"Length: {len(set_with_caps)}")
print("\nObservation: Strings with different capitalization are treated as
different values.")
```

```
# Numbers with different types
set_numbers = {1, 1.0, 1, 2, 2.0, 2}
print(f"\nSet with numbers: {set_numbers}")
print("Note: 1 and 1.0 are considered the same in Python sets")
```

Output:

```
Set with different capitalizations:
Input: {'apple', 'Apple', 'APPLE', 'Banana', 'banana', 'BANANA'}
Output: {'apple', 'BANANA', 'APPLE', 'Banana', 'banana', 'Apple'}
Length: 6
Observation: Strings with different capitalization are treated as different values.
Set with numbers: {1, 2}
Note: 1 and 1.0 are considered the same in Python sets
```

Cell 96: ■ Markdown

Conclusion

Hence, we have studied and implemented data structures in Python. The key learnings include:

1. **Comments**: Used to explain code and improve readability
2. **Primitive Data Types**: Integer, Float, String, Complex, Boolean, Bytes
3. **Lists**: Ordered, mutable collections with various operations
4. **Tuples**: Ordered, immutable collections
5. **Dictionaries**: Key-value pairs for efficient data retrieval
6. **Sets**: Unordered collections of unique elements
7. **Control Flow**: If-else statements for decision making

8. **Loops**: For loops for iterating over collections

9. **Functions**: Reusable blocks of code with parameters and return values

Python's data structures provide flexible and efficient ways to organize and manipulate data for various programming tasks.

Cell 97: ■ Markdown

Name:

SAP ID:

Signature of Faculty: